



Prof. Adriano Silva
adrianovss@gmail.com

adrianovss@gmail.com

Ementa

- Correção da Atividade 01 + Revisão
- Estruturas de Dados (listas, tuplas e dicionários)
- Estruturas de Fluxo (Repetição)
 - For
 - While
- *List Comprehension*
- Funções

Ementa (+)

- **Módulos e Pacotes (Parte 01)**
- **Map, Filter, Zip**
- **Lambda**



Correção da Atividade 01 + Revisão



E continuando...

Listas (*list*)

- Listas são grupos de itens ou elementos indexados, não necessariamente do mesmo tipo;
- São **mutáveis**, isso significa que seu conteúdo pode ser alterado sem que um novo objeto seja criado;
- Os índices em uma lista sempre se iniciam em 0 (zero);
- Listas podem conter sub listas que podem conter sub listas e assim por diante, tornando esse tipo de dado extremamente versátil;
- Pode se usar operações como slicing ([], [:]), concatenação (+), repetição (*) e membership (in).

Funções e métodos mais comuns com o tipo *list*

Função	Descrição
<code>len(lista)</code>	Retorna o tamanho total da lista
<code>max(lista)</code>	Retorna o maior elemento da lista
<code>min(lista)</code>	Retorna o menor elemento da lista
<code>list(tupla)</code>	Converte uma tupla em uma lista
<code>list.append(obj)</code>	Adiciona um objeto à lista
<code>list.insert(index, obj)</code>	Insere um objeto à lista em determinada posição
<code>list.count(obj)</code>	Retorna a quantidade de vezes que um determinado objeto ocorre na lista
<code>list.index(obj)</code>	Retorna o primeiro índice de ocorrência de um determinado objeto
<code>list.remove(obj)</code>	Remove um objeto da lista
<code>list.reverse()</code>	Reverte o ordenamento da lista
<code>list.sort()</code>	Ordena a lista

Tuplas (*tuple*)

- Tuplas são tipos muito semelhantes às listas, porém **imutáveis**;
- Por esse motivo, são mais eficientes que as listas - não existe a necessidade de alteração de seu conteúdo;
- Para atribuir elementos em uma tupla é necessário que os mesmos estejam entre parênteses e separados por vírgula. **Ex.:** $t = (item1, item2, \dots itemn)$;
- Caso a tupla possua apenas um elemento, será necessário adicionar uma vírgula após a declaração do mesmo. **Ex.:** $t = (item1,)$;
- Índices funcionam da mesma forma das listas, assim como os métodos e funções.

Dicionários (*dict*)

- Consiste em uma lista de pares de **chave** x **valor**, sem ordenação;
- Chaves, devem ser tipos imutáveis e usualmente são utilizadas strings ou números;
- Valores podem ser de qualquer tipo de objeto;
- Exemplo: $dic = \{ 'k1': 1, 'k2': 10, \dots 'kn': n \}$
- Os elementos de um dicionário podem ser acessados ou alterados através de sua chave entre colchetes. **Ex.:** $dic['k1'] = 2$

Funções e métodos mais comuns com o tipo *dict*

Função	Descrição
<code>len(dict)</code>	Retorna o número de elementos do dicionário
<code>str(dict)</code>	Retorna a representação em formato string do dicionário
<code>dict.keys()</code>	Retorna a lista de chaves do dicionário
<code>dict.values()</code>	Retorna a lista de valores do dicionário
<code>dict.items()</code>	Retorna a lista de chave, valor do dicionário
<code>dict.get(key, default=None)</code>	Retorna o valor de uma chave ou um valor default caso não seja encontrada
<code>dict.update(dict2)</code>	Insere um elemento (chave, valor) no dicionário
<code>dict.clear()</code>	Remove todos os elementos do dicionário

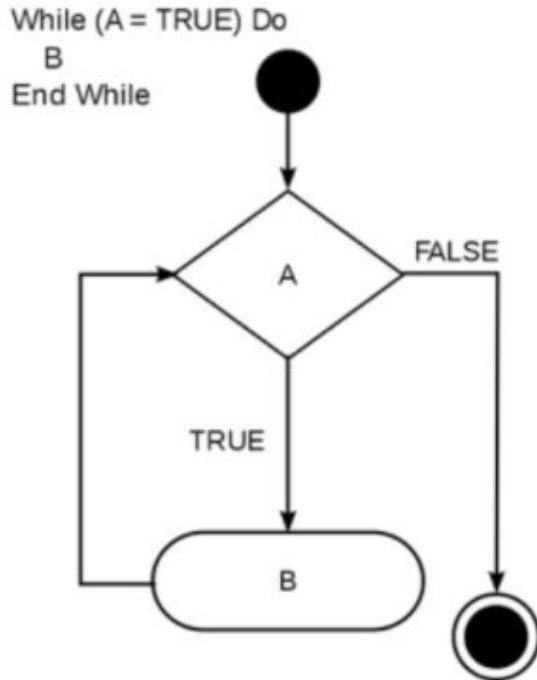
Exercícios



Estruturas de Fluxo (Repetição)

- Estruturas de Repetição são utilizadas para instruir o programa a repetir um determinado conjunto de código enquanto uma determinada condição seja satisfatória;
- São utilizadas também para iterar em coleções de dados ou listas;
- O **for** é mais utilizada na iteração de listas ou quando há um número conhecido de iterações a serem executadas;
- O **while** é mais recomendado quando uma determinada condição seja satisfatória para a execução do trecho de código.

Instruções de Controle



- **break**: permite a saída do fluxo antes da condição de finalização do laço ser atingida;
- **continue**: instrui o interpretador a passar para a próxima iteração imediatamente, ignorando os comandos subsequentes;
- **pass**: permite que um determinado trecho de código seja escrito sintaticamente correto, porém em sua execução não será executada nenhuma instrução naquele momento.

List Comprehension

- Para cada valor de uma lista, aplica-se uma operação e adicionam-se os resultados em uma nova lista.

Do this	For this collection	In this situation
<code>[x**2</code>	<code>for x in range(0, 50)</code>	<code>if x % 3 == 0]</code>

Exercícios



Funções

- Simplificadamente, uma função é um conjunto de instruções do qual pode-se dar um nome;
- Funções são muito úteis para organizar o código;
- Promovem o reaproveitamento, uma vez que um podem ser chamadas em diversos pontos do mesmo programa, sem a necessidade reescrita desse trecho de código;
- A identificação é fundamental para que o interpretador identifique quais são de fato, as instruções que pertencem à função.

Funções (continuação...)

- Os parâmetros de uma função podem ser passados na mesma ordem em que foram definidos ou podem ser identificados através da função chamadora;
- Parâmetros de funções podem ter valores **default**, dessa forma não é necessário sua passagem como argumento (desde sejam os últimos argumentos em ordem reversa);
- Funções suportam parâmetros com número indefinido de valores. Para isso é necessário que sejam o último parâmetro da função e o argumento seja precedido por um * (asterisco);
- Funções podem retornar valores, através da instrução **return**. Não é necessário declarar que a função irá ter um retorno.

Exercícios





Facens

AQUI TEM ENGENHARIA